Supply, Not Selection: What Limits Sequential Reciprocal Matching in the Vouchsafe Simulator

Team: Laplace · **Author:** Kumar Ankit · Technical report for the final build

All people, preferences and outcomes are synthetic. Every number below comes from the public simulator, through the scripts in `scripts/`. The results describe this invented environment, not real relationships.

1. Summary

Claim tested. In Round 1 I proposed that an online-learned reciprocal value model, ranked clarification and global matching would beat the supplied baselines. I also listed three questions to test:

1. What does online adaptation add beyond a weak fixed prior?
2. How much does the approximate matcher lose against an exact general-graph solver?
3. Are the directional and joint estimates calibrated, and does the scarcity adjustment help?

Main finding. Clarification is the only lever with a large, reliable effect on the primary outcome. The value model raises mutual acceptances, but the gain does not survive the noisy post-acceptance steps. The other components have little room to work. In these worlds the feasible graph is very thin: each member with any feasible partner has about 2–3 over the whole episode. Every policy that clarifies ends up introducing most feasible pairs within the 60 days. MSMI is then decided mostly by which pairs exist and by pair-level noise, not by which pair is chosen first.

Answers to the three questions.

- On MSMI, the following are statistically indistinguishable from their simpler alternatives: online adaptation, the exact solver and the scarcity adjustment (fresh reporting seeds), and Thompson sampling, soft asks and greedy-vs-global matching (validation seeds).
- The approximate matcher is essentially optimal on the graphs the policy actually faces.
- The estimates are reasonably calibrated in aggregate but only weakly discriminative.

The submitted policy is the simplified configuration chosen on validation seeds: online learning, posterior mean, ranked clarification and global matching, with Thompson sampling, soft asks and evidence discounting removed. §8 says plainly which simpler policy would score the same.

Headline numbers (fresh reporting seeds 101–120 × 6 variants, 120 episodes per policy, all valid). The submitted configuration was fixed on validation seeds before these runs. The submitted policy scores **0.404 MSMI per 100 arrived members** [95% CI 0.325, 0.492], against greedy 0.433, random-feasible 0.367 and no-clarification 0.183, with coverage 0.344. The MSMI differences among clarifying policies are not significant. On mutual acceptances, however, the submitted policy is ahead: **5.81 vs 5.19 per 100** for greedy (paired +0.62 [+0.35, +0.89]) and 5.52 for random (paired +0.29 [+0.03, +0.55]). It also gets more dates (+0.45 [+0.16, +0.73] vs greedy). On validation seeds that intermediate gain was not reproduced for the final configuration, so I treat it as modest and possibly seed-dependent.

2. Decision problem and policy

Objective. Maximise mean MSMI per 100 arrived members across the six scenario families. That means maximising the expected number of introductions that reach mutual acceptance, a date within 30 days, and two second-meeting Yes answers within 3 days. All this happens under the hard rules, in the order the primary score and tie-breaks set.

Hard vs soft (W3 workshop principle).

- **Hard** (define the feasible set, never traded off): reciprocal eligibility via `kit.eligibility`, availability, one introduction per person per batch, no repeated pair, and the daily ask budget.
- **Soft** (rank feasible actions): predicted pair value, coverage, waiting and ask cost.

A pair with a missing hard field is never proposed. A *declined* field blocks a member permanently: such members are never asked again and never inferred.

Layers. These are kept separate (PS §15):

1. `observe.py` builds the day's feasible graph and the matured labels.
2. `model.py` estimates values.
3. `asks.py` and `matching.py` decide.
4. `core.py` validates every action and falls back to an empty, valid action if anything fails.

Clarification (`asks.py`).

- *Constraint bundles (3 units):* only for available members with missing but not declined hard fields. They are ranked by *unlock value*: the number of other members not already ruled out by a known hard failure, counting complete partners as 1 and still-askable partners as 0.4.

- *Soft fields (1 unit each), from the leftover budget:* the highest-|learned weight| unknown fields of members in, or about to enter, the candidate graph. Implemented, but **off in the submission**: validation showed no benefit, and it would raise ask cost from about 197 to about 556.

Pair value (`model.py`). For a feasible pair (a, b) with three-state soft-field agreement g (+1 agree, −1 disagree, 0 unknown):

- $p_{ab} = \sigma(w_0 + w \cdot g + u_a)$ is P(a says Yes | a responds). u_a is a member effect.
- ρ_a is a's response propensity: Beta-binomial shrinkage toward the episode's pooled response rate.
- $q_{ab} = \sigma(c \cdot g)$ is P(MSMI | mutual Yes).
- The value is $V_{ab} = (\rho_a \rho_b)^2 p_{ab} p_{ba} q_{ab}$. Each person must respond twice: to the introduction and to the second-meeting question.

Multiplying p_{ab} by p_{ba} assumes the two directions are conditionally independent; §6.3 tests this.

Allocation (`matching.py`). Each day, a maximum-weight matching on the general graph with edge weights $V_{ab} \cdot (1 + s \cdot (1/\deg_a + 1/\deg_b)/2)$:

- The *scarcity adjustment*, s = 1, favours members with few feasible options.
- The algorithm is greedy followed by local improvement: augmenting through free vertices and 2-edge swaps until no move gains.
- A waiting threshold (shadow price of a person's next slot) was implemented and tested, but is off by default (§6).

3. Observations, censoring and leakage

The policy reads only the request: `state` and `memory`. Memory holds just the soft-field agreement vector and the predictions *at assignment time* for each introduced pair. That way the online learner never attaches later-clarified profile values to earlier outcomes (PS §9). Memory is a few tens of kB, against a 1 MiB limit.

Label rules (`observe.labels`):

- A directional label exists only once a Yes or No is observed.
- A deadline with no response updates only ρ, and is never a preference label (PS §8).
- A pair's success label exists only once its window has closed:
- **0** if no date occurred within 30 days, or the date failed, or more than 3 days passed after the date without two timely Yes answers.
- **1** if both timely Yes answers arrived.
- Pending outcomes are censored, i.e. omitted.

Unit tests (`test_laplace.py`) check these rules, action validity in all six variants, memory size, and the matcher.

4. Training, assets, randomness and dependencies

- **Prior asset** (`laplace/priors.json`): fitted by `scripts/fit_priors.py` on declared training seeds 1000–1059, all variants, 360 episodes.
- The rollouts ran this policy from zero-mean priors.
- Training rows are decision-time agreement vectors plus matured observed feedback: 41,964 directional responses and 4,046 matured mutual acceptances, with 308 MSMI events.
- No hidden simulator field, seed or generator coefficient is read (PS §11; FAQ).
- The asset only sets prior means. Online fitting uses prior s.d. 0.6, roughly 20 pseudo-observations per weight.
- **Seeds:** validation 2000–2019 for design decisions; reporting 101–120 for every table in §5–§7. The reporting seeds were never used for tuning.
- **Determinism:** Thompson draws use `random.Random(1000003·day + 17)`, so the policy is a deterministic function of the state.
- The fast in-process harness (`scripts/fast_eval.py`) reproduces `evaluate.py` exactly; identical action SHA-256 digests were checked on sampled episodes.
- Final reporting numbers were also confirmed through `evaluate.py` (§5).
- **Dependencies:** inference uses the standard library only, in the `python:3.12.7-slim` image. SciPy (HiGHS MILP) is used only offline for the exact matching reference.
- **Compute:** an 8-core laptop CPU. One subprocess episode takes about 20 s of policy time (about 0.17 s per call). A container episode takes about 126 s, mostly `docker run` start-up (§5).
- **Tools:** AI coding assistance (Anthropic's Claude, via Claude Code) was used to write code, run experiments and draft this report. I reviewed the design decisions and every reported number is reproducible from the committed scripts.

5. Results against the supplied baselines (reporting seeds 101–120)

All six variants, 20 fresh seeds each, run with identical seeds for every policy. The MSMI/100 interval resamples seeds within each variant. The full machine-readable episodes are in `results/reporting/`.

run	MSMI/100 [95% CI]	mutual/100	dates/100	assign/100	coverage	unserved	median 1st wait	ask cost	missing fb	infer s/ep	valid
submitted	0.404 [0.325, 0.492]	5.81	4.60	37.2	0.344	131.2	3.0	197	38.3	7.8	120/120
greedy	0.433 [0.354, 0.517]	5.19	4.16	37.7	0.345	131.0	4.5	200	38.9	9.8	120/120
random	0.367 [0.300, 0.433]	5.52	4.28	37.6	0.345	131.1	4.6	200	39.1	6.2	120/120
no_asks	0.183 [0.138, 0.233]	1.73	1.34	10.4	0.123	175.4	0.2	0	10.5	8.4	120/120
online_off	0.362 [0.292, 0.438]	5.96	4.66	37.2	0.344	131.2	3.1	197	37.6	7.1	120/120
scarcity_off	0.375 [0.304, 0.454]	5.89	4.56	37.4	0.344	131.2	3.0	197	37.7	8.0	120/120
exact_matcher	0.429 [0.350, 0.513]	5.82	4.60	37.4	0.345	131.0	3.1	197	38.3	8.4	120/120
reference_full_info	0.467 [0.396, 0.542]	5.97	4.67	37.6	0.345	131.0	0.0	0	38.3	8.0	120/120

MSMI per 100 arrived members by variant (mean of 20 seeds)

run	development	sparse	cold_start	delayed	shift	drift
submitted	0.525	0.075	0.450	0.500	0.350	0.525
greedy	0.675	0.075	0.300	0.375	0.550	0.625
random	0.500	0.075	0.275	0.375	0.525	0.450
no_asks	0.325	0.000	0.100	0.150	0.200	0.325
online_off	0.500	0.100	0.375	0.300	0.450	0.450
scarcity_off	0.500	0.075	0.450	0.425	0.325	0.475
exact_matcher	0.575	0.075	0.450	0.500	0.425	0.550
reference_full_info	0.600	0.025	0.600	0.575	0.400	0.600

Mutual acceptances per 100 by variant

run	development	sparse	cold_start	delayed	shift	drift
submitted	7.33	1.82	5.25	6.97	6.53	6.95
greedy	6.30	1.70	5.35	6.28	5.65	5.88
random	6.62	1.70	5.60	6.42	6.60	6.17
no_asks	2.25	0.53	1.02	2.35	2.00	2.23
online_off	7.33	1.85	5.65	7.05	6.95	6.95
scarcity_off	7.50	1.82	5.40	6.97	6.78	6.88
exact_matcher	7.22	1.85	5.35	7.15	6.53	6.83
reference_full_info	7.38	1.75	5.62	7.45	6.50	7.10

Paired differences: submitted minus run (positive = submitted better), 95% CI

run	ΔMSMI/100	Δmutual/100	Δdates/100	Δcoverage
greedy	-0.029 [-0.125, +0.067]	+0.617 [+0.354, +0.887]*	+0.446 [+0.158, +0.725]*	-0.001 [-0.002, +0.000]
random	+0.037 [-0.058, +0.129]	+0.288 [+0.029, +0.546]*	+0.329 [+0.038, +0.625]*	-0.001 [-0.002, +0.001]
no_asks	+0.221 [+0.150, +0.292]*	+4.079 [+3.812, +4.358]*	+3.263 [+3.017, +3.521]*	+0.221 [+0.214, +0.228]*
online_off	+0.042 [-0.017, +0.104]	-0.154 [-0.350, +0.054]	-0.054 [-0.237, +0.129]	-0.000 [-0.001, +0.000]
scarcity_off	+0.029 [-0.000, +0.062]	-0.083 [-0.229, +0.058]	+0.042 [-0.079, +0.175]	-0.000 [-0.001, +0.000]
exact_matcher	-0.025 [-0.062, +0.008]	-0.012 [-0.108, +0.079]	+0.004 [-0.100, +0.104]	-0.001 [-0.002, -0.001]*
reference_full_info	-0.062 [-0.154, +0.029]	-0.158 [-0.487, +0.167]	-0.062 [-0.354, +0.208]	-0.001 [-0.002, +0.000]

`*` interval excludes zero.

Outcome funnel for submitted by variant (means per episode)

variant	arrived	assignments	mutual	dates	MSMI	coverage	unserved	missing fb
development	200.0	87.8	14.65	11.65	1.05	0.389	122.3	45.9
sparse	200.0	21.8	3.65	2.80	0.15	0.170	165.9	10.4

variant	arrived	assignments	mutual	dates	MSMI	coverage	unserved	missing fb
cold_start	200.0	74.2	10.50	8.15	0.90	0.337	132.7	39.1
delayed	200.0	85.7	13.95	11.20	1.00	0.390	122.1	44.0
shift	200.0	89.1	13.05	10.40	0.70	0.390	122.0	44.0
drift	200.0	88.2	13.90	11.05	1.05	0.389	122.2	46.2

Harness and container checks.

- `evaluate.py` in subprocess mode on reporting seed 101 (all six variants) reproduced the fast harness's action digests 6/6 (`results/evaluate_submitted_seed101.json`).
- Container mode (`evaluate.py --image sequential-policy:laplace --seeds 101`, offline, 2 CPU, 1 GiB) produced a valid episode with an identical digest. Policy time was 1.05 s per call including `docker run` start-up, against the 10 s limit (`results/container_check.json`).

What the table shows.

- **Clarification is the one large effect.** Without it, coverage falls from 0.34 to 0.12, and MSMI falls by 55% on reporting seeds and 77% on validation seeds.
- **Among policies that clarify** (greedy, random, submitted and every ablation), MSMI differences are inside the bootstrap intervals.
- **On mutual acceptances and dates**, the submitted value model beats greedy (+12%) and random (+5%) on these seeds, with intervals that exclude zero. Those extra acceptances did not turn into more MSMI: the post-acceptance steps (date, two timely Yes answers) are dominated by noise the policy cannot observe.
- **Event counts are small.** There are roughly 90–110 MSMI events per 120-episode run, so a 10% effect is not detectable at this sample size. The same is true of the validation set (§6).

6. Design evidence: what each component adds

All design decisions were taken on validation seeds 2000–2019, using paired bootstrap intervals (`results/validation/analysis.md`).

Validation, paired against the Round 1 design (positive = Round 1 design better):

Change	ΔMSMI/100 [95% CI]	Δmutual/100 [95% CI]
Thompson sampling off	−0.079 [−0.129, −0.029]	+0.038 [−0.167, +0.242]
Soft asks off	−0.062 [−0.117, −0.008]	−0.004 [−0.163, +0.150]
No evidence discount	−0.033 [−0.062, −0.004]	−0.054 [−0.163, +0.054]
Online learning off (fixed weak prior)	−0.042 [−0.100, +0.021]	+0.025 [−0.171, +0.233]
Online learning from a zero prior	−0.029 [−0.083, +0.025]	−0.013 [−0.163, +0.133]
Baseline (list-order) asks	+0.000 [−0.100, +0.104]	+0.058 [−0.263, +0.371]
Scarcity adjustment off	−0.012 [−0.050, +0.025]	+0.021 [−0.083, +0.121]
Greedy instead of local-improvement matcher	−0.008 [−0.046, +0.029]	−0.042 [−0.171, +0.075]
Waiting threshold on (κ = 0.5)	−0.017 [−0.054, +0.021]	−0.154 [−0.287, −0.029]
Cardinality-first matching	−0.025 [−0.058, +0.004]	−0.092 [−0.217, +0.029]
All three "winners" combined (`final`)	−0.033 [−0.083, +0.021]	+0.242 [+0.029, +0.463]

How the submission was chosen.

- Three single changes looked better on MSMI: turning off Thompson sampling, turning off soft asks, and removing the evidence discount.
- Combining them did *not* reproduce the best single gain (0.417 vs 0.463 MSMI/100). It also lost mutual acceptances. Across 14 comparisons this is what regression to the mean looks like, so I treat the individual "wins" as noise.
- The submission therefore follows a parsimony rule, not a score rule:
- Thompson sampling off: there is almost no real choice to explore.
- Soft asks off: no benefit, and lower ask cost is the fourth ranking tie-break.
- No discount: simpler.
- Online learning on: no cost, and a hedge if the private engine differs.
- The **first design (v1)** asked soft fields only when a candidate partner already showed them, so about 500 budget units per episode went unused. Validation: 0.417 MSMI/100, 5.19 mutual/100. A broad soft-ask rule spent the budget but changed nothing (0.383, 5.27).

6.1 Q1: what does online adaptation add beyond a weak fixed prior?

On reporting seeds, the fixed weak prior (`online_off`) scored 0.362 against 0.404 MSMI/100: a paired difference of +0.042 [−0.017, +0.104] in favour of online learning, which is not significant. Mutual acceptances moved the other way: −0.154 [−0.350, +0.054]. Validation agreed: online off was +0.042 [−0.021, +0.100] for the Round 1 design.

The mechanism works. End-of-horizon acceptance weights move away from the prior in the direction each variant implies:

- In *shift*, the lifestyle weight flips sign, from +0.094 to −0.076, and relationship pace rises from +0.29 to +0.42, while relationship goal falls (+0.38 to +0.31).
- In the stationary families, goal rises (≈ +0.53) and weak fields shrink toward zero.

(`results/reporting_submitted_calibration.md`.)

It does not change outcomes, because learning has little to act on.

- The feasible graph per episode is small: in development seed 2003, 109 feasible pairs among 84 members, mean degree 2.6.
- Every clarifying policy makes about 74 introductions per episode, so most feasible pairs get used whichever estimate ranks them.
- Better estimates change the *order*, not the *set*, of introductions. That order matters only when a high-degree member's time binds.

The honest answer to Q1: online adaptation estimates the shifted preferences correctly, but in these worlds it adds no measurable MSMI over a weak fixed prior.

6.2 Q2: approximate vs exact general-graph matching

`scripts/matching_gap.py` compares the shipped matcher (greedy + local improvement) with an exact 0/1 integer program (SciPy HiGHS, verified against brute force on 200 small graphs). The exact solver is never in the image.

Graphs	Mean edges	Greedy / exact (mean, min)	Local / exact (mean, min)	Local optimal
Real daily decision graphs (360, validation seeds)	4.3	0.999, 0.912	**1.000, 1.000**	100%
Random, 120 vertices, density 0.01	72	0.994, 0.958	0.999, 0.993	90%
density 0.02	143	0.983, 0.957	0.993, 0.959	25%
density 0.05	354	0.972, 0.941	0.986, 0.975	0%
density 0.10	711	0.976, 0.960	0.984, 0.960	0%
density 0.20	1431	0.972, 0.950	0.980, 0.960	0%
density 0.50	3567	0.974, 0.961	0.983, 0.970	0%

Allocation value. As density grows, the local matcher loses about 1–2% of total weight (worst case 4%), and greedy loses about 2.5–3%. On the graphs this policy actually faces, the local matcher is optimal every time: the graphs are tiny because most members are busy or exhausted on any given day. Runtime stays below 0.25 s even at 3,567 edges.

Final outcomes. Running the exact MILP solver *inside* the policy on reporting seeds gave 0.429 against 0.404 MSMI/100 (paired −0.025 [−0.062, +0.008]), mutual −0.012 [−0.108, +0.079], and coverage +0.001. That is no detectable outcome difference, consistent with the matcher already being optimal on those graphs. Tiny tie-breaking differences change which pairs are used later in the episode, and that is the source of the noise. Swapping the local matcher for plain greedy changed validation MSMI by −0.008 [−0.046, +0.029].

Recommendation. If a future engine produced much denser daily graphs, an exact stdlib blossom implementation would be worth porting, at a 1–2% allocation gain. Here it is not.

6.3 Q3: calibration, and whether the scarcity adjustment helps

Decision-time predictions were stored in policy memory and scored against matured outcomes on reporting seeds (`results/reporting_submitted_calibration.md`).

Target	n	Mean predicted	Observed	Brier (constant-rate)	ECE
Directional P(Yes | responded)	13752	0.484	0.471	0.2497 (0.2492)	0.048
Joint P(both Yes | both responded) = p_ab·p_ba	5295	0.248	0.263	0.1943 (0.1940)	0.052
Reciprocal P(MSMI) = pair value v	8935	0.008	0.011	0.0107 (0.0107)	0.005

Directional estimates. Well calibrated in the middle of the range, but overconfident at the top: the highest decile predicts 0.70 and observes 0.55. Discrimination is modest: the observed rate rises from about 0.41 to 0.55 across deciles.

Joint estimates. The product of directional estimates is *not* an adequate joint model. The two directions' residuals are positively correlated within a pair (+0.16), so a shared pair component exists. The product is too low for low-value pairs (0.08 predicted vs 0.15 observed) and too high for the top decile (0.49 vs 0.37). A pair-level random effect or a direct joint model is the fix.

Reciprocal outcome. P(MSMI) is under-predicted on average, at 0.8 vs 1.1 per 100 introductions, and its Brier score barely beats a constant rate. The pair value ranks pairs only weakly, which is consistent with §6.1: most of the outcome variance is pair-level noise the observable fields do not explain.

Scarcity adjustment. It boosts edges of members with few options. On reporting seeds, removing it changed MSMI by −0.029 [−0.000, +0.062] (borderline, in the direction of the adjustment helping), mutual acceptances by +0.083 [−0.058, +0.229], and coverage by less than 0.001. On validation it changed MSMI by +0.012 [−0.025, +0.050] and coverage by less than 0.001. It neither helps nor hurts after calibration, because nearly every member with a feasible partner is served anyway. Unserved members are blocked by declined fields or have no feasible partner, and no reordering can change either cause.

7. Failure analysis

Who is left waiting (submitted policy, reporting seeds, per episode):

Group	Members	Coverage	Median first wait	90th percentile
Arrived on day 0	140.3	0.347	5 days	16 days
Arrived days 1–20	59.7	0.336	0 days	11 days

Unserved members by cause (per episode, out of about 131 unserved):

- **67.9 are blocked by a declined hard field.** These can never be introduced; the policy never asks them and never infers the answer.
- **62.3 have complete hard fields but were never introduced.** Almost all of them have no reciprocally feasible partner at any point.
- **1.1 still had unclarified hard fields at the end.** These are mostly late arrivals who were unavailable when the budget was spent.

Late arrivals are not disadvantaged: their coverage is similar (0.336 vs 0.347) and their median first wait is 0 days.

Where introductions end (per episode):

Outcome	Count
A missing response	30.3
An explicit No	32.5
No date, or the date failed	2.4
Date after 30 days	0.07
Second-meeting answer missing, late or No	8.3
MSMI	0.81

The 3-day second-meeting window and response noise remove most mutually accepted pairs. The policy cannot act on either.

By scenario family.

- *Sparse* (12 single-zone geographies) is the hardest: there are about 12 feasible pairs per world, and every policy uses all of them.
- *Cold start* raises clarification cost and waits, but clarification still finishes within days.
- *Delayed*: across all families, only 0.07 introductions per episode had a date after the 30-day window, so the main loss in *delayed* is not the window itself.
- In *shift*, online weights adapt (§6.1).
- In *drift*, the late acceptance penalty affects all policies equally.

Validity. Every run, on validation and reporting seeds, had 0 invalid episodes. The tests check validity in all six variants. The container check is reported in §5. Policy time is about 0.1 s per call in subprocess mode, against the 10 s limit.

8. What the comparison taught me, and limits

1. **Measure supply before modelling preference.** The binding constraint is the reciprocal feasible graph and the 8-day lock-up, not pair selection. A policy that clarifies everyone who can be clarified and then introduces any feasible pair each day captures nearly all of the attainable MSMI. On these worlds that policy is the supplied greedy or random-feasible baseline plus clarification, and it scores the same as mine within noise.
2. **A simpler method works as well on the primary score.** The greedy baseline, which counts equal soft fields, is statistically tied with the submitted policy on MSMI. The value model buys about 12% more mutual acceptances on reporting seeds (not reproduced on validation), but this does not reach the final outcome. My extra machinery is justified mainly by robustness and auditability:
3. a validated, crash-safe action path;

4. decision-time predictions logged for calibration;
5. weights that visibly track preference shifts.

It is not justified by score. 3. **Pre-registered gates and paired tests prevented false claims.** Three "significant" single-component wins did not combine (§6). Without a confirmation run I would have overstated them. 4. **The analysis-only full-information reference** reveals every disclosed answer at arrival, in the harness only. It scored 0.467 MSMI/100 and 5.97 mutual/100, against 0.404 and 5.81 for the same policy under normal observation (paired −0.062 [−0.154, +0.029], not significant), with identical coverage (0.345 vs 0.344). Missing information is therefore not the bottleneck either. Declined answers and thin reciprocal supply are.

Limitations.

- Outcome events are rare: about 0.8 MSMI per episode. Effects below about 15% of the primary score cannot be resolved with 120 episodes per policy.
- The joint acceptance model ignores the positive within-pair dependence it detected.
- All conclusions concern this invented simulator. Its generator draws member biases, pair noise and response rates that no observable field explains, so it says nothing about real people.

Product use. Later authorised evaluation would need the following (docs/INTEGRATION.md):

- a validated adapter from production records to this observation contract;
- current eligibility checks;
- human review of every introduction;
- recalibration on authorised data.

References

1. M. Akbarpour, S. Li, S. Oveis Gharan. Thickness and Information in Dynamic Matching Markets. *JPE* 128(3):783–815, 2020.
2. R. Anderson, I. Ashlagi, D. Gamarnik, Y. Kanoria. Efficient Dynamic Barter Exchange. *Operations Research* 65(6):1446–1459, 2017.
3. J. Edmonds. Paths, Trees, and Flowers. *Canadian J. Math.* 17:449–467, 1965.
4. D. E. Drake, S. Hougardy. A Simple Approximation Algorithm for the Weighted Matching Problem. *IPL* 85:211–213, 2003.
5. O. Chapelle, L. Li. An Empirical Evaluation of Thompson Sampling. *NIPS* 2011.
6. H. Lakkaraju et al. The Selective Labels Problem. *KDD* 2017.
7. R. A. Howard. Information Value Theory. *IEEE Trans. SSC* 2(1):22–26, 1966.
8. Programme workshops W1–W3 (exploration/exploitation; robust ML from messy data; hard rules, soft goals), as credited in the Round 1 note.